

# Розробка клієнта для взаємодії з віддаленою папкою з файлами. Етап 4, варіант 4-6

Порівняльний аналіз десктоп- і веб-версій

Виконав: студент групи МІ-41 Петров Богдан

Перевірив: \_\_\_\_\_

Київський національний університет імені Тараса Шевченка  
Дисципліна «Інформаційні технології»  
Київ — 2026

## Зміст

|          |                                             |          |
|----------|---------------------------------------------|----------|
| <b>1</b> | <b>Мета порівняння</b>                      | <b>1</b> |
| <b>2</b> | <b>Спільні елементи</b>                     | <b>2</b> |
| <b>3</b> | <b>Відмінності</b>                          | <b>2</b> |
| <b>4</b> | <b>Переваги та недоліки кожної версії</b>   | <b>5</b> |
| <b>5</b> | <b>Обґрунтування оптимальної технології</b> | <b>6</b> |
| <b>6</b> | <b>Висновки</b>                             | <b>7</b> |

## 1 Мета порівняння

На етапі 2 реалізовано десктоп-клієнт (Electron) і сервер (NestJS), на етапі 3 — веб-клієнт (Next.js) для тієї самої системи MiniDrive. Обидва клієнти реалізують один і той самий функціональний контракт варіанта 4-6 (реєстрація/вхід, перегляд файлів власного простору, сортування за назвою, фільтр «лише .cpp» / «лише .png», перегляд .cs як тексту й .jpg як зображення, завантаження/скачування/видалення, синхронізація локальної папки) поверх одного й того самого REST API. Мета цього звіту — порівняти дві реалізації клієнта за архітектурою, обсягом коду, платформними обмеженнями та експлуатаційними властивостями, і на основі порівняння обґрунтувати, яка технологія є оптимальною для системи такого типу (тип 2 — клієнт-серверна система з віддаленим сховищем файлів).

## 2 Спільні елементи

Десктоп- і веб-версії — не дві незалежні реалізації, а один клієнт, розгорнутий у двох середовищах виконання:

- **Сервер і REST-контракт.** Обидва клієнти звертаються до одного інстансу NestJS API за одними й тими самими маршрутами (POST /auth/register, POST /auth/login, GET /auth/me, GET /files, POST /files, GET /files/:id/content, DELETE /files/:id) з однаковими форматами відповідей (AuthResponseDto, FileDto) і кодами помилок (401, 404, 409, 413, 400). Зміна серверної логіки одразу відображається в обох клієнтах без додаткової синхронізації коду.
- **packages/shared.** Типи (FileDto, UserDto, AuthResponseDto), ApiClient (обгортка над fetch), чисті функції варіанта sortByName/filterByType, previewKindOf/createPreview, toggleColumn, фасад стану DriveViewModel і правила синхронізації computeSyncPlan/reconcileWithLedger — увесь цей код написано один раз без залежностей від Electron, DOM чи Node-специфічних API і використовується без жодної зміни і в apps/desktop, і в apps/web.
- **packages/ui.** Презентаційні React-компоненти DriveWorkspace і LoginForm (а також FileTable, SortControl, FilterControl, ColumnToggle, PreviewPanel, UploadDropzone, SyncPanel) рендерять однаковий інтерфейс в обох клієнтах; усе платформозалежне (виклик API, збереження сесії, вибір папки, drag-out) передається всередину пропсами чи callback-ами, а не імпортується напряму з electron чи next/\*. SyncPanel сама володіє станом busy/progress/report/error; кожен клієнт лишає собі лише тонку обгортку (apps/desktop/src/renderer/src/components/SyncPanel.tsx, apps/web/src/components/SyncPanel.tsx), що постачає в неї pickFolder і runSync — платформозалежний вибір папки та власне запуск синхронізації (IPC до головного процесу на десктопі, BrowserSyncEngine у браузері).
- **Тести доменної логіки.** sortByName, filterByType, previewKindOf, toggleColumn, DriveViewModel, computeSyncPlan покриті одним і тим самим набором тестів у packages/shared, тож обидва клієнти успадковують однакову перевірену поведінку сортування, фільтрації, перегляду й синхронізації.
- **Той самий стек.** TypeScript у строгому режимі, React 19, Tailwind/shadcn-стилі, Yarn 4 workspaces (workspace:\* для внутрішніх пакетів), однакова українська термінологія інтерфейсу («Увійти», «Зареєструватися», «Оновити», «Скачати», «Видалити», «Вийти», «Усі файли», «Лише .crrp», «Лише .png», «Синхронізувати», «Обрати папку»).

## 3 Відмінності

| Критерій                   | Десктоп (Electron)                                                                                                                                     | Веб (Next.js)                                                                                                                                                                                                                                 |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Платформа запуску          | Нативний застосунок (Chromium + Node у власному процесі)                                                                                               | Будь-який сучасний браузер, сторінка на <a href="https://DOMAIN">https://DOMAIN</a>                                                                                                                                                           |
| Зберігання сесії           | JWT в Electron safeStorage (шифрування ОС, apps/desktop/src/main/settings.ts)                                                                          | JWT у localStorage (apps/web/src/lib/session.ts, apps/web/src/lib/minidrive.token)                                                                                                                                                            |
| Доступ до файлової системи | Node fs/fs/promises напряду з головного процесу                                                                                                        | File System Access API (showDirectoryPicker, apps/web/src/lib/browserSync.ts) — лише Chrome/Edge                                                                                                                                              |
| Синхронізація: облік стану | Локальний mtime файлу зіставляється з updatedAt сервера                                                                                                | localStorage-реєстр (SyncLedger, apps/web/src/lib/syncLedgerStore.ts), бо File System Access API не дає керувати mtime                                                                                                                        |
| Автоматичне відстеження    | FolderWatcher на chokidar (дебаунс 2 с) запускає синхронізацію на кожну зміну в папці                                                                  | Відсутнє — синхронізація лише за кнопкою «Синхронізувати»                                                                                                                                                                                     |
| Drag-and-drop              | Двонапрямний: drop у вікно — завантаження (UploadDropzone), перетягування рядка з таблиці назовні — скачування (DragOutHandler, WebContents.startDrag) | Лише drop у вікно для завантаження; перетягування рядка назовні неможливе технічно (посилання на скачування вимагає заголовка Authorization: Bearer, а системний drag браузера не може додати HTTP-заголовок до запиту, який він сам ініціює) |
| Скачування файлу           | Нативний діалог збереження (dialog.showSaveDialog через win-dow.minidrive.file.saveAs)                                                                 | Завантаження браузером у папку завантажень за замовчуванням                                                                                                                                                                                   |
| Пакування й розповсюдження | .dmg/.zip через electron-builder, підписання коду вимкнено (identity: null, сертифіката розробника немає)                                              | Docker-образ (minidrive-prod-web, docker/web.Dockerfile) за Caddy з автоматичним TLS, доступ за URL                                                                                                                                           |

| Критерій                                    | Десктоп (Electron)                                                                                                                                                                                                                                       | Веб (Next.js)                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Оновлення клієнта                           | Користувач вручну<br>перевстановлює нову збірку                                                                                                                                                                                                          | Миттєве — нове розгортання<br>образу підхоплюється всіма<br>користувачами одразу при<br>наступному запиті                                                                                                                                                                                                                                                                           |
| Підтримка браузерів/ОС                      | Кросплатформно<br>(macOS/Windows/Linux —<br>окрема збірка electron-builder<br>на кожную), але один<br>застосунок на пристрій                                                                                                                             | Будь-яка ОС з підтримуваним<br>браузером; повна<br>функціональність<br>(синхронізація папки) лише в<br>Chrome/Edge через File System<br>Access API, Firefox/Safari —<br>без синхронізації папки                                                                                                                                                                                     |
| Безпека виконання                           | Типова ізоляція процесів<br>Electron (контекстна ізоляція<br>та вимкнена інтеграція Node у<br>рендерері за замовчуванням) і<br>звужений PreloadBridge,<br>локальний доступ до файлів<br>обмежено<br>isSafeFileName/перевіркою<br>виходу за межі каталогу | Same-origin політика браузера,<br>TLS на транспорті (Caddy/Let's<br>Encrypt); production API<br>дозволяє довільне джерело<br>(CORS_ORIGINS=*), бо<br>десктоп-клієнт надсилає<br>Origin: null і API все одно<br>автентифікує лише за<br>Beaeger-токеном без cookie;<br>XSS у веб-клієнті потенційно<br>небезпечніший, бо токен<br>лежить у localStorage,<br>доступному з JS сторінки |
| Продуктивність запуску                      | Повільніший холодний старт<br>(запуск процесу<br>Electron/Chromium), швидша<br>робота після старту (Node<br>напрямку)                                                                                                                                    | Миттєвий запуск (відкриття<br>вкладки), рендер через мережу<br>при першому завантаженні<br>сторінки                                                                                                                                                                                                                                                                                 |
| Обсяг коду (рядки, без тестів)              | apps/desktop/src — 674                                                                                                                                                                                                                                   | apps/web/src — 339                                                                                                                                                                                                                                                                                                                                                                  |
| Обсяг спільного коду                        | packages/shared/src — 355,<br>packages/ui/src — 1507<br>(спільно для обох клієнтів)                                                                                                                                                                      | те саме                                                                                                                                                                                                                                                                                                                                                                             |
| Залежності (dependencies у<br>package.json) | 7 прямих (electron-store,<br>chokidar,<br>@electron-toolkit/* тощо) +<br>Electron у devDependencies                                                                                                                                                      | 6 прямих (next, react,<br>react-dom, lucide-react,<br>@minidrive/shared,<br>@minidrive/ui)                                                                                                                                                                                                                                                                                          |

| Критерій            | Десктоп (Electron)                                                                                                                      | Веб (Next.js)                                                                                   |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Розмір дистрибутиву | MiniDrive-1.0.0-arm64.dmg<br>— 241 176 673 байт (≈230 МБ),<br>.zip — 233 209 086 байт (≈222 МБ), розпакований<br>MiniDrive.app — 615 МБ | Docker-образ<br>minidrive-prod-web — 282 МБ (70 МБ унікальних шарів, решта — спільна база Node) |

Обсяг коду в цифрах (рядки `.ts/.tsx/.css`, без `.test./spec.`, виміряно `git ls-files ... | xargs wc -l` у цьому репозиторії):

| Область             | Рядків |
|---------------------|--------|
| packages/shared/src | 355    |
| packages/ui/src     | 1507   |
| apps/api/src        | 662    |
| apps/desktop/src    | 674    |
| apps/web/src        | 339    |

Частка спільного клієнтського коду: `packages/shared` + `packages/ui` = 355 + 1507 = 1862 рядки проти `apps/desktop/src` + `apps/web/src` = 674 + 339 = 1013 рядків платформоспецифічного коду. Тобто приблизно 65 % коду, що формує UI та логіку обох клієнтів, написано один раз і використовується двічі; лише 35 % — код, специфічний для конкретної платформи (Electron-обв'язка з одного боку, Next.js-обв'язка з іншого). Показово, що `apps/web/src` (339 рядків) майже вдвічі менший за `apps/desktop/src` (674 рядки) — головна причина в тому, що вебу не потрібні `preload`-міст, IPC-обробники та нативні діалоги, які в Electron написані вручну поверх Node/Chromium.

Автоматизовані тести: `packages/shared` — 51, `apps/api` — 23, `apps/desktop` — 7, `apps/web` — 4, разом 85 тестів, усі проходять (`yarn test` з кореня репозиторію). Невелика кількість тестів у `apps/web` пояснюється тим, що вся логіка, яку є сенс тестувати ізольовано (правила синхронізації, сортування, фільтрація, перегляд), уже покрита тестами `packages/shared`; тести `apps/web` перевіряють лише специфічну для браузера частину — `BrowserSyncEngine/peer SyncLedger`.

## 4 Переваги та недоліки кожної версії

### Десктоп (Electron)

Переваги: - Автоматична синхронізація папки без участі користувача (`FolderWatcher` на `chokidar`). - Повноцінний двонаправний `drag-and-drop` (завантаження і скачування перетягуванням). - Нативні діалоги вибору файлу/папки і збереження, звичні користувачеві macOS/Windows/Linux. -

Токен сесії зберігається в `safeStorage`, зашифрованому засобами ОС. - Працює без залежності від конкретного браузера чи його версії.

Недоліки: - Великий дистрибутив (сотні мегабайт на кожную ОС) і потреба вручну встановлювати й оновлювати застосунок. - Збірка без підписання коду (немає сертифіката розробника) — на macOS/Windows користувач отримає попередження системи безпеки при першому запуску. - Окрема збірка `electron-builder` для кожної ОС (`build:mac/build:win/build:linux`), більше поверхні для підтримки.

### Веб (Next.js)

Переваги: - Нульове встановлення: доступ за URL у будь-якому сучасному браузері. - Миттєве оновлення — нове розгортання образу відразу доступне всім користувачам без переустановлення. - Один Docker-образ і один сервер розгортання замість збірок під кожную ОС. - TLS з коробки через Caddy й Let's Encrypt.

Недоліки: - Немає автоматичного відстеження змін у папці — синхронізація лише вручну, за кнопкою. - File System Access API (а отже й сама можливість синхронізації папки) підтримується лише в Chrome/Edge; у Firefox і Safari `SyncPanel` недоступна. - Drag-out (перетягування файлу з таблиці назовні вікна) неможливий технічно через потребу в `Authorization`-заголовку для скачування. - Токен у `localStorage` вразливіший до XSS, ніж токен у `safeStorage` десктоп-клієнта. - Синхронізація не може виставити `mtime` локального файлу (браузер цього не дозволяє), тому використовується окремий реєстр (`SyncLedger`) — додаткова умовність порівняно з прямим підходом десктопа.

## 5 Обґрунтування оптимальної технології

Для системи типу 2 (клієнт-серверна робота з віддаленим файловим простором, у якій ключова цінність — доступ до файлів «звідусіль» і мінімальний поріг входу) оптимальною основною технологією є **веб-версія**: нульове встановлення, розгортання одного стека (один Docker-образ, один сервер), той самий код інтерфейсу (`packages/ui`) і доменної логіки (`packages/shared`), що й у десктопі, а отже й та сама якість і ті самі 47+4 тести покривають фактичну поведінку користувача. Веб-версія — правильний вибір для більшості користувачів і для першого знайомства з системою.

Водночас **десктоп-версія залишається виправданою там, де потрібні дві речі, недосяжні у браузері**: автоматична фонові синхронізація локальної папки (`chokidar`-спостереження без участі користувача) і повноцінний `drag-out` (перетягування файлу з таблиці в Finder/Провідник). Обидва обмеження веб-версії — не недогляд реалізації, а технічні межі веб-платформи (`File System Access API` не дає слідкувати за директорією, а посилання на завантаження не можуть нести Bearer-токен у системному `drag` браузера).

Завдяки `packages/ui` і `packages/shared` вартість підтримки обох версій одночасно невелика: 1862 рядки спільного коду проти 1013 рядків платформоспецифічного, тобто лівова частка інтерфейсу й логіки написана й протестована один раз. Тому раціональна стратегія — веб як основний канал

доступу для всіх користувачів, десктоп як опційний додатковий клієнт для тих, кому потрібна автоматична синхронізація папки або drag-out, без дублювання зусиль розробки.

## 6 Висновки

Десктоп- і веб-клієнти MiniDrive побудовані на спільному фундаменті (`packages/shared`, `packages/ui`, один REST API) і відрізняються лише в тонкому платформозалежному шарі: зберігання сесії, доступ до файлової системи, спосіб синхронізації та пакування. Виміряні дані підтверджують, що ця спільність — не декларація, а факт коду: 65 % рядків клієнтського інтерфейсу й логіки написано один раз для обох платформ. Відмінності, що лишаються (автоматичне відстеження папки, двонаправний drag-and-drop, спосіб зберігання токена), впливають із меж можливостей веб-платформи, а не з різного рівня реалізації. Для системи типу 2 оптимальна стратегія — веб-версія як основний канал (нульове встановлення, миттєве оновлення, один стек розгортання), десктоп-версія — як додатковий клієнт для сценаріїв, що потребують автоматичної синхронізації та drag-out; сервер на момент написання звіту підготовлено до розгортання (`docker/compose.prod.yml`, Caddy з автоматичним TLS), але ще не опубліковано в інтернеті — публічна адреса буде додана після розгортання користувачем на VM.